

PROJE 03 / 20 · ÇALIŞAN TEMEL SÜRÜM

Markdown Blog ve CMS

Markdown taslakları hazırla, güvenli önizle ve çalışan düzenlemeleri fark ederek yayımla.

Yerel içerik yönetimi

Bu proje nedir?

Frontend kullanıcının gördüğü ekranları ve etkileşimleri, backend iş kurallarını ve veri erişimini yürütür. Bu uygulama ikisini aynı origin üzerinde birleştiren bağımsız bir full-stack örnektir. Üretim ürünü değil, üzerinde geliştirme yapabileceğin çalışan başlangıçtır.



Gerçek uygulama ekranı · yerel demo verileri

Teslim edilen kapsam

Taslak oluşturma ve düzenleme · Temizlenmiş HTML önizleme · Yayımlama ve public JSON · Version ile iyimser kilitleme · Yazı sahipliği kontrolü

Bu sürümde olmayanlar

Public uç nokta JSON döndürür; ayrı herkese açık blog sayfası ve SEO render yok. Görsel yükleme, etiket ve revizyon geçmişi sonraki sürüm. Slug değişmez; kısa UUID soneki kullanılır.

02 / Backend ve veri modeli

Markdown parser çıktısı sanitize-html ile temizlenir. Önizleme ve public endpoint aynı render fonksiyonunu çağırır. Güncelleme WHERE version=? koşuluyla yapılır ve version artırılır; sıfır satır değişirse 409 döner. Yazı yayımlanınca taslak değil published durumuna geçer. Kimlik bilinmesi yazı sahipliği kontrolünü atlatmaz.

Gerçek SQL şeması

```
CREATE TABLE posts(id TEXT PRIMARY KEY,owner TEXT REFERENCES users(id),title TEXT,slug TEXT UNIQUE,markdown TEXT,status TEXT DEFAULT 'draft',version INTEGER DEFAULT 1,updated_at TEXT);
```

Tam şema: schema.sql. users ve sessions ortak kimlik tablolarıdır. owner/user alanları kullanıcıyı, PRIMARY KEY kaydı, UNIQUE tekiliği ve REFERENCES ilişkiyi belirtir. Para kullanılan projelerde INTEGER kuruş, zaman alanlarında ISO metin veya epoch kullanılır.

Kod okuma sırası

Dosya	Ne arayacaksın?
client/Workspace.vue	Eylemi tetikleyen form ve API isteği
client/api.ts	JSON isteği, hata yakalama ve mesaj
server/project.js	Yetki kontrolü → kural → SQL → yanıt
server/core.js	Parametrelili SQL, transaction, oturum, SSE

Neden bu mimari?

Aynı dilde frontend/backend öğrenmeyi kolaylaştırır. Node.js 24 ve Express HTTP katmanıdır; Vue arayüzü yönetir. SQLite ayrıca sunucu kurmadan kalıcılık sağlar. Senkron veritabanı işlemleri küçük yerel demo için seçildi; çok süreçli, yüksek trafikli üretim mimarisi iddiası yok.

03 / API sözleşmesi

Aşağıdaki yollar doğrudan teslim edilen server/project.js dosyasından çıkarılmıştır. JSON alan adları kodla aynıdır. :id ve benzeri path alanları oluşturma yanıtından alınmalıdır. Liste/görünüm uçlarında 200, yeni kayıt uçlarında 201, kuyruk kabulünde 202 kullanılır.

Yöntem / yol	Girdi alanları	Başarı
GET /public/posts/:slug	Gövde yok	200
GET /api/posts	Gövde yok	200
POST /api/preview	JSON: markdown	200
POST /api/posts	JSON: title, markdown	201
PATCH /api/posts/:id	JSON: title, markdown, version	200
POST /api/posts/:id/publish	Gövde yok	200
DELETE /api/posts/:id	Gövde yok	200

Somut istek örneği

```
POST /api/posts
{"title":"Vue notları","markdown":"# Merhaba"}
201 {"id":"<post-id>","slug":"vue-notlar-..."}

```

Oturum ve hata biçimi

POST /api/auth/login: email ve password. Başarılı giriş HttpOnly çerez verir. GET /api/auth/me kullanıcıyı, POST /api/auth/logout çıkışı yönetir. Tarayıcı aynı origin kullanır. Genel hata biçimi: {"error":"açıklama"}. Yetkisiz alan işlemleri sunucuda engellenir; frontend kontrolü tek başına yeterli değildir.

Kod	Anlam
401 / 403	Giriş gerekli / rol veya origin reddedildi
404 / 409	Kayıt görünmüyor / durum veya benzersizlik çakışması
422 / 429	Geçersiz girdi / giriş veya uygulama kotası
500 / 502 / 503	Beklenmeyen hata / dış servis hatası / kapalı veya ayarsız servis

Tüm uçlar bütün kodları üretmez. İstek/yanıt gövdelerini tests/project.test.js ve scripts/seed.js içindeki çalışan örneklerle karşılaştır.

04 / Test ve kabul kriterleri

Bir projenin çalışması yalnız ekrana veri gelmesi değildir. Yetki, tekrar istek, yanlış girdi ve yarış koşulları da beklenen sonucu üretmelidir. Aşağıdaki maddeler teslim edilen temel sürümün kontrol listesidir.

- Taslak public endpoint üzerinden okunamamalı.
- Eski version ile yazma 409 dönmeli.
- Script ve javascript: bağlantısı önizlemede etkin olmamalı.
- Başka kullanıcı yazıyı silememeli.

Otomatik test nasıl çalışır?

npm test, Node test runner ile gerçek HTTP sunucusunu rastgele yerel portta başlatır. İki kullanıcıyla oturum açar, uç noktalara istek gönderir ve durum/yanıtları assert ile denetler. Veritabanı :memory: olduğu için kendi data/app.sqlite dosyan etkilenmez. Projeye özgü testler tests/project.test.js içindedir.

```
npm ci
npm run typecheck
npm test
npm run build
```

Test kodundan örnek

```
const p=await post('/posts',{title:'Test',markdown:'# Hello'},201);
await call('/public/posts/'+p.slug,'GET',undefined,404);
const row=(await get('/posts'))[0];await
call('/api/posts/'+p.id,'PATCH',{title:'New',markdown:'Good',version:row.version},200);
await
call('/api/posts/'+p.id,'PATCH',{title:'Old',markdown:'Lost',version:row.version},409);
await call('/api/posts/'+p.id,'DELETE',undefined,404,other);
```

Tarayıcıda kontrol et

Bir başarılı işlem, bir hatalı girdi ve boş durum göster. 390 px mobil görünümde yatay taşmayı kontrol et. Sayfayı yenilediğinde kalıcı kayıtlar durmalı. Network panelinde isteğin gövdesini ve dönen HTTP kodunu oku; başarısız işlemi başarılıymış gibi göstermemelisin.

Doğrulamanın sınırı

API entegrasyon testleri, üretim derlemesi ve Vue tip/şablon kontrolü uygulandı. Bunlar yük testi, penetrasyon testi veya tüm tarayıcılarda kapsamlı uçtan uca test yerine geçmez.

05 / Çalışma ve GitHub planı

İlk günün somut hedefi

Bir taslak oluştur ve iki sekmede aç. İlk sekmede kaydet; ikinci sekmede eski version ile kaydın neden reddedildiğini Network panelinde incele.

Çalışma bloğu	Teslim edeceğin çıktı
1 · Kurulum / 30-60 dk	Çalışan ekran, örnek veri ve ilk başarılı API isteği
2 · Kod okuma / 45-90 dk	Formdan SQL sorgusuna kadar bir isteğin açıklaması
3 · Geliştirme / 60-120 dk	Bir iş kuralı veya arayüz davranışında kendi değişikliğin
4 · Doğrulama / 30-60 dk	İlgili test ve README değişikliği; yeni ekran görüntüsü

Süreler tahmindir; hata çözme ve yeni kavramlar daha uzun sürebilir. Bu plan hazır temel sürüm üzerinde çalışmak içindir. Projeyi sıfırdan ve üretim kalitesinde bir günde bitirme vaadi değildir.

İleri sürüm: dört gerçek geliştirme

- 1. Okuyucu için public sayfa ekle
- 2. Etiket ve arama ekle
- 3. Revizyon geçmişini ayrı tabloya taşı
- 4. Sunucu render ve SEO meta verilerini ekle

Anlamli commit sırası

```
git add .
git commit -m "chore: add markdown-cms baseline"
# Kendi değişikliğini yaptıktan sonra:
git add client server tests README.md
git commit -m "feat: describe your actual improvement"
```

Yeni boş GitHub reposuna remote ekleyip push et. Burada gerçek hesap/repo adresi bulunmadığı için otomatik yükleme yapılmadı. node_modules, data ve .env dosyalarını hariç tut. Her gün bir gerçek ilerleme paylaş; commit sayısını artırmak için geçmiş üretme.

Mülakatta anlat

Optimistic locking hangi durumda daha uygundur? Markdown parser kullanmak neden HTML temizliğinin yerine geçmez?

Özel kurulum / ilerleme notu

Önce yerel sürümü tamamla. Sonraki aşamada PostgreSQL, migration, pagination ve gerçek kullanıcı yönetimini ihtiyaca göre ekle. Ortak kurulum ayrıntıları README.md ve başlangıç rehberindedir.